

FIE463 Term Paper 2

Expectations about macroeconomic and financial variables

```
In [1]: # --- 1. Master Environment Components & Library Dependencies ---
# It is structural coding best practice to universally lock all dependency modul

# Standard Core Libraries
import glob          # WS 9 for å matche filer med et mønster/pattern
import os            # WS 9 for operasjone
import warnings      # Usikker på om har blitt brukt tidligere, men jeg har br

# Datastrukturer, vektorisering og grafisk fremstilling
import matplotlib.pyplot as plt
import numpy as np   # Mange WS's, Lecture 9-12
import pandas as pd  # Masse i WS's og

# Scikit-Learn
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LinearRegression, RidgeCV, LassoCV
from sklearn.linear_model import LogisticRegressionCV
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_sc
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.model_selection import GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

# Warnings
warnings.filterwarnings('ignore') # Tar bort innebygde warnings i output fra fek
```

Part 1 - Data preprocessing (SCE)

```
In [2]: # Definerer filsti
DATA_DIR = './data'

# for å bruke glob (WS 9)
file_pattern = os.path.join(DATA_DIR, "sce_extract_*.csv")
file_list = glob.glob(file_pattern)

# Bruker pd.concat for å merge en lite med dataframes (WS 9!!)
df_sce = pd.concat([pd.read_csv(f) for f in file_list], ignore_index=True)

# Oversikt
print(f"Successfully merged {len(file_list)} files.")
print(f"Total initial rows: {len(df_sce)}")
print(f"Dataset columns:\n {df_sce.columns.tolist()}")
```

Successfully merged 13 files.

Total initial rows: 180268

Dataset columns:

```
['userid', 'wid', 'tenure', 'weight', 'date', 'financial_past_12m', 'financial_12m', 'prob_move_house', 'prob_unrate_up', 'prob_irate_up', 'prob_stocks_up', 'infl_1y', 'infl_1y_bin_mean', 'infl_1y_bin_var', 'infl_1y_bin_median', 'infl_1y_bin_iqr', 'infl_1y_bin_prob_defl', 'infl_3y', 'infl_3y_bin_mean', 'infl_3y_bin_var', 'infl_3y_bin_median', 'infl_3y_bin_iqr', 'infl_3y_bin_prob_defl', 'infl_5y', 'working', 'num_jobs', 'self_employed', 'prob_lose_job', 'prob_leave_job', 'looking_for_job', 'prob_accept_job_12m', 'prob_accept_job_3m', 'jobless_length', 'prob_search_job_12m', 'prob_search_job_3m', 'earnings_change', 'hh_inc_change', 'hh_spending_change', 'taxes_change', 'credit_cond_past_12m', 'credit_cond_12m', 'prob_miss_paym_3m', 'house_price_change', 'house_price_change_3y', 'govt_debt_change', 'num_lit_q1', 'num_lit_q1_correct', 'num_lit_q2', 'num_lit_q2_correct', 'num_lit_q3', 'num_lit_q3_correct', 'num_lit_q5', 'num_lit_q5_correct', 'num_lit_q6', 'num_lit_q6_correct', 'num_lit_q8', 'num_lit_q8_correct', 'num_lit_q9', 'num_lit_q9_correct', 'age_init', 'female', 'hispanic', 'black', 'educ', 'owner', 'health', 'take_fin_risk', 'hh_changed', 'num_kids', 'same_employer', 'couple', 'hh_inc_bin', 'hh_inc_bin_rank']
```

In [3]: *# 1.2 Forward-fill variables asked only in the initial wave*

```
df_sce['date'] = pd.to_datetime(df_sce['date'])
df_sce = df_sce.sort_values(by=['userid', 'date'])

# OBS SE SENERE: 'health' og 'take_fin_risk' har veldig mange missing values. Dr
# Når jeg tar med faller sample til ~123k. Å overse de fører oss til ~150k !!!
initial_wave_vars = ['owner', 'educ', 'female', 'num_lit_q1_correct', 'num_lit_q2_correct', 'num_lit_q3_correct']

for col in initial_wave_vars:
    if col in df_sce.columns:
        df_sce[col] = df_sce.groupby('userid')[col].ffill()

print(f"Successfully forward-filled the following initial-wave variables:\n{initial_wave_vars}")
```

Successfully forward-filled the following initial-wave variables:

```
['owner', 'educ', 'female', 'num_lit_q1_correct', 'num_lit_q2_correct', 'num_lit_q3_correct']
```

1.3 Drop Year 2025 Survey Waves

As specified in the assignment, we filter out any 2025 observations to circumvent the irregular variance and noise introduced contextually in the dataset.

In [4]: *# 1.3 Drop 2025*

```
# Storing total rows before drop for reporting (Required by Part 1 guidelines)
rows_before_drop2025 = len(df_sce)

# LECTURE 9 TRIKS: Filtering out 2025 using pandas boolean indexing on the date
df_sce = df_sce[df_sce['date'].dt.year < 2025]

# Using a standard pandas copy to prevent SettingWithCopyWarning down the line
df_sce = df_sce.copy()

print(f"Rows before dropping 2025: {rows_before_drop2025}")
print(f"Number of observations dropped (2025): {rows_before_drop2025 - len(df_sce)}")
print(f"Total remaining rows: {len(df_sce)}")
```

Rows before dropping 2025: 180268
Number of observations dropped (2025): 4167
Total remaining rows: 176101

1.4 Clean NAs

We isolate the specific features and targets used in our machine learning pipeline and subsequently drop any observations missing data points inside this core subset. As before, we report the exact count of observations dropped.

```
In [5]: # 1.4 Drop rows with missing data for variables used in analysis
rows_before_na_drop = len(df_sce)

# Define our continuous modeling targets
expectation_vars = ['infl_1y', 'house_price_change', 'prob_unrate_up', 'prob_sto

# Define essential identifying variables
id_vars = ['userid', 'wid', 'date']

# Calculate final core subset, using initial_wave_vars defined in 1.2
core_columns = id_vars + expectation_vars + initial_wave_vars

# Ensure we only evaluate columns that actually exist in the dataframe
available_cols_to_check = [col for col in core_columns if col in df_sce.columns]

# Drop NaNs subsetted specifically to our modeling variables
df_sce = df_sce.dropna(subset=available_cols_to_check).copy()

# Retain only these columns to optimize memory utilization for performance
df_sce = df_sce[available_cols_to_check]

print(f"Rows before dropping NaNs: {rows_before_na_drop}")
print(f"Number of observations dropped (NaNs): {rows_before_na_drop - len(df_sce)}")
print(f"Total remaining rows: {len(df_sce)}")
```

Rows before dropping NaNs: 176101
Number of observations dropped (NaNs): 3268
Total remaining rows: 172833

1.5 Handle Outliers

To improve the robustness and generalization of our continuous variable machine learning models (e.g. Ridge and Lasso regressors later on), we explicitly truncate observations falling outside the 1st and 99th percentile bounds for each of the four main expectation factors. We report the exact amount of dropped outliers as well.

```
In [6]: # 1.5 Handle Outliers (strict clipping between P1 and P99)

rows_before_outliers_drop = len(df_sce)

for col in expectation_vars:
    # Compute the 1st (P1) and 99th (P99) percentiles using pandas
    p1 = df_sce[col].quantile(0.01)
    p99 = df_sce[col].quantile(0.99)

    # Filter dataset sequentially keeping only values strictly > P1 and < P99
```

```
df_sce = df_sce[(df_sce[col] > p1) & (df_sce[col] < p99)]

print(f"Rows before handling outliers: {rows_before_outliers_drop}")
print(f"Number of outlier observations dropped: {rows_before_outliers_drop - len(df_sce)}")
print(f"Total remaining rows: {len(df_sce)}")
```

Rows before handling outliers: 172833
 Number of outlier observations dropped: 17740
 Total remaining rows: 155093

1.6 & 1.7 Feature Engineering (Binary Indicators) & Final Report

We convert continuous expectation expectations into a binary format (1 for Optimistic view, 0 for Pessimistic view) to be used as discrete targets for classification modeling in Part 6. Finally, we run an output summary to validate the final cleaned SCE table before proceeding to Part 2.

```
In [7]: # 1.6 Create binary indicator variables

df_sce['optimist_unrate'] = np.where(df_sce['prob_unrate_up'] < 50, 1, 0)
df_sce['optimist_stocks'] = np.where(df_sce['prob_stocks_up'] > 50, 1, 0)
df_sce['optimist_house_price'] = np.where(df_sce['house_price_change'] > 0, 1, 0)

# Final Part 1 Preprocessing Report
# EGEN CODE BLOCK ELLER?
print("--- Final Cleaned SCE Dataset ---")
print(f"Total Unique Individuals: {df_sce['userid'].nunique()}")
print(f"Total Observations: {len(df_sce)}")
print(f"Number of Survey Waves: {df_sce['date'].nunique()}")
print(f"Starting Date: {df_sce['date'].min().strftime('%Y-%m-%d')}")
print(f"Ending Date: {df_sce['date'].max().strftime('%Y-%m-%d')}")
```

```
--- Final Cleaned SCE Dataset ---
Total Unique Individuals: 22021
Total Observations: 155093
Number of Survey Waves: 4106
Starting Date: 2013-06-01
Ending Date: 2024-12-31
```

Part 2 - Macroeconomic & Financial Data

To predict expectations without violating causality (look-ahead bias), we fetch raw macro data, strictly resample it to Month-End (ME), and execute a `merge_asof` algorithm with `direction='backward'` so respondents only see data theoretically published *prior* or strictly *on* their survey date.

```
In [8]: # 2.1 Load and process macro/finance time-series

# Map of filenames to their (Raw Target Column, Standardized Lowercase Column)
# Brukte yFinance, men tar ikke med i koden siden det ikke skal lastes inn på ny
macro_files = {
    'CPIAUCSL.csv': ('CPIAUCSL', 'cpi'),
    'UNRATE.csv': ('UNRATE', 'unrate'),
    'GDPC1.csv': ('GDPC1', 'gdp'),
    'MORTGAGE30US.csv': ('MORTGAGE30US', 'mortgage'),
```



```

    'FEDFUNDS.csv': ('FEDFUNDS', 'fedfunds'),
    'USSTHPI.csv': ('USSTHPI', 'house_idx'),
    'GSPC.csv': ('Close', 'sp500'),
    'DTWEXBGS.csv': ('DTWEXBGS', 'dollar'),
    'CL_F.csv': ('Close', 'oil')
}

df_macro_list = []

# Iterating to keep our code DRY
for file, (val_col, new_name) in macro_files.items():
    filepath = os.path.join(DATA_DIR, file)
    if os.path.exists(filepath):
        df_temp = pd.read_csv(filepath)

        # Ensure we only pick the Date and the appropriate Value column
        df_temp = df_temp[['Date', val_col]].rename(columns={'Date': 'date', val_col: new_name})

        # Convert to datetime and cast any non-numeric coercions seamlessly
        df_temp['date'] = pd.to_datetime(df_temp['date'])
        df_temp[new_name] = pd.to_numeric(df_temp[new_name], errors='coerce')

        # Sort values and drop localized NAs
        df_temp = df_temp.sort_values('date').dropna()

        # Standardize Frequency to Month End ('ME') using .last() method
        df_temp = df_temp.set_index('date').resample('ME').last()

        df_macro_list.append(df_temp)

# Concat horizontally using the datetime index mathematically aligned
df_macro = pd.concat(df_macro_list, axis=1)

# Forward-fill any NaN overlaps organically caused by differing publication schedule
df_macro = df_macro.ffill()

# Reset index so 'date' is a column again for merge_asof
df_macro = df_macro.reset_index()

print("Resampled Macro Headers:")
print(df_macro.columns.tolist())

```

Resampled Macro Headers:

```
['date', 'cpi', 'unrate', 'gdp', 'mortgage', 'fedfunds', 'house_idx', 'sp500', 'dollar', 'oil']
```

2.2 Calculate Trailing Dynamics & Merge

Respondents react to momentum, not just flat values. We calculate the 1-month and 12-month trailing absolute differences and percentage changes for all macroeconomic indicators, then execute the strictly backward `merge_asof` operation.

```

In [9]: # 2.2 Calculate 1-month / 12-month changes and merge

macro_vars = [col for col in df_macro.columns if col != 'date']

# Create trailing differences and percentage changes
for col in macro_vars:
    # 1-month momentum

```

```

df_macro[f"{col}_diff1"] = df_macro[col].diff(1)
df_macro[f"{col}_pct1"] = df_macro[col].pct_change(1)

# 12-month momentum
df_macro[f"{col}_diff12"] = df_macro[col].diff(12)
df_macro[f"{col}_pct12"] = df_macro[col].pct_change(12)

# It's explicitly required to sort both dataframes chronologically by the 'on' k
df_sce = df_sce.sort_values('date')
df_macro = df_macro.sort_values('date')

initial_size = len(df_sce)

# Backward ASOF Merge: Guarantees no Look-ahead data Leakage! (From Workshops)
# Takes the survey date, and looks purely backward in the df_macro matrix to sim
df_final = pd.merge_asof(
    df_sce,
    df_macro,
    on='date',
    direction='backward'
)

# Because diff(12) requires 12 months of historical data to compute,
# some rows in df_final might contain NaNs in their trailing columns depending o
# We drop these as they represent our mathematical 'burn-in' period.
df_final = df_final.dropna(subset=[f"{macro_vars[0]}_diff12"])

# We must now reset our tracking sizes because we've shifted variables significa
print(f"SCE shape before merge: {initial_size}")
print(f"Rows dropped due to 'burn-in' trailing macro calculation: {initial_size}
print(f"Final merged dataset size ready for modeling: {len(df_final)}")

```

SCE shape before merge: 155093

Rows dropped due to 'burn-in' trailing macro calculation: 0

Final merged dataset size ready for modeling: 155093

2.3 Economic Motivation for Feature Engineering Format Restrictions

We deliberately structure our trailing macroeconomic regressors into three distinct mathematical formats, aligning with behavioral economics principles regarding how households functionally evaluate the state of the economy:

1. Raw Levels (e.g., Unchanged Rates) Raw levels establish the *baseline context* or anchor for the respondent. An individual's fear of unemployment realistically increasing is heavily mathematically dependent on the current baseline condition. For instance, expecting unemployment to rise when the current base level is 3.5% represents a significantly different psychological environment than expecting it to rise when the base is naturally anchored at 9.0%.

2. Absolute Differences (1-month & 12-month) Absolute differences are the correct method to measure momentum for variables that are *already expressed as percentages or rates* (like FEDFUNDS interest rates or the Unemployment Rate). If the Fed Funds rate jumps from 1.0% to 2.0%, that is cleanly quantified as a +1.0% absolute point jump. If we incorrectly computed a relative percentage change for this momentum block, it would

register as an extreme +100% surge, which artificially exaggerates the data and distorts linear regression sensitivity boundaries.

3. Relative Percentage Changes (1-month & 12-month) Raw values for massive economic indices (like the S&P 500 index, real GDP level, or the abstract dollar indices) are non-stationary and functionally useless to the average consumer. A consumer doesn't shape their personal expectations systematically based on whether the S&P 500 is trading at exactly 4,000 points or 4,100 points; they base it on the contextual news headline that "the stock market is up 20% this year." By strictly computing 1-month and 12-month percentage shifts, we transform non-stationary index evaluations into stationary, normalized variables that replicate how households perceive and measure performance constraints.

Part 3 - Exploratory Data Analysis (SCE)

We generate exploratory visualizations to review the statistical distributions of our continuous expectation variables prior to regression modeling.

3.1 Histograms of Expectation Responses

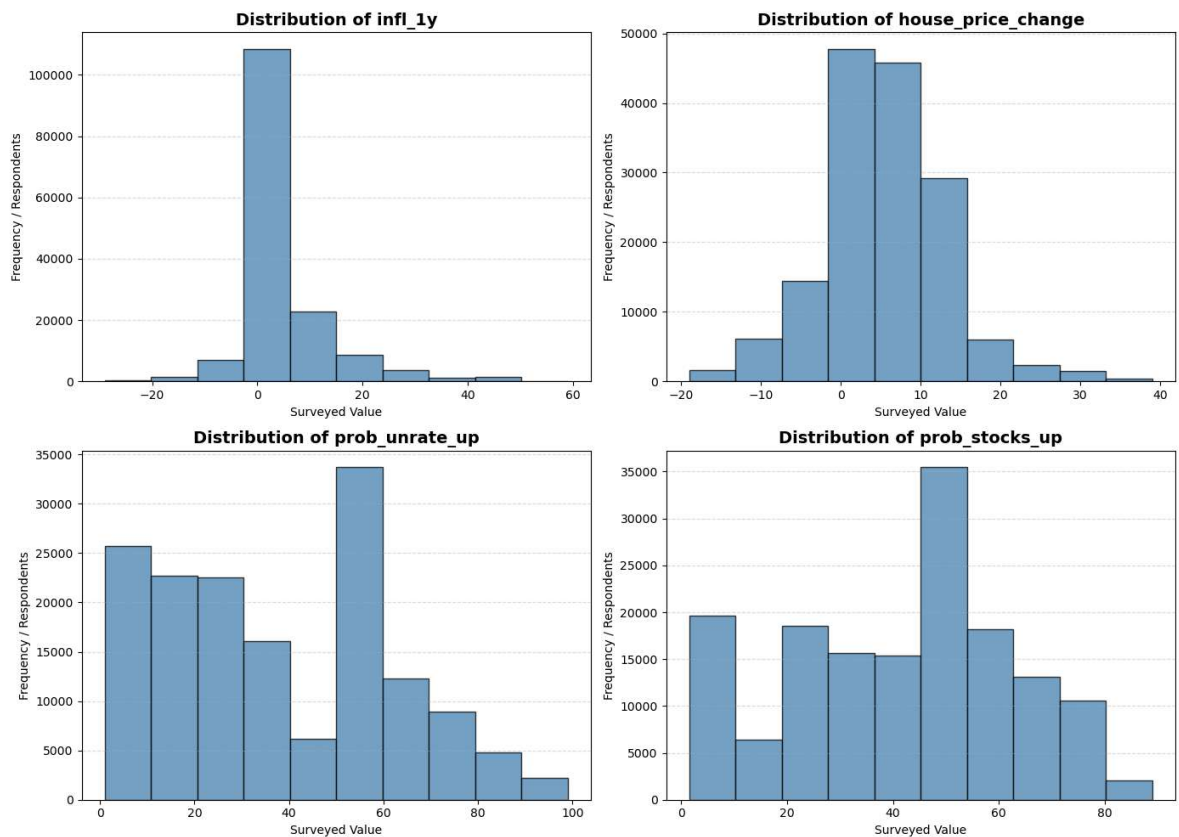
We map the mass distribution for inflation expectations, house price changes, unemployment probability, and stock market probability using Matplotlib.

```
In [10]: # 3.1 Histograms of continuous targets

# Using a streamlined subplot matrix for cleaner notebook rendering (Concept head)
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
axes = axes.flatten()

# expectation_vars defined earlier: ['infl_1y', 'house_price_change', 'prob_unra
for i, col in enumerate(expectation_vars):
    # Generating 10 bins strictly against valid target values
    axes[i].hist(df_sce[col].dropna(), bins=10, color='steelblue', edgecolor='bl
    axes[i].set_title(f'Distribution of {col}', fontsize=14, fontweight='bold')
    axes[i].set_ylabel('Frequency / Respondents')
    axes[i].set_xlabel('Surveyed Value')
    axes[i].grid(axis='y', alpha=0.4, linestyle='--')

plt.tight_layout()
plt.show()
```



3.2 Time Series of Averages & Optimist Fractions

We aggregate our granular panel dataset into singular monthly timestamps utilizing pandas `.groupby('date').mean()`. This naturally outputs both the global average for continuous indicators, and natively calculates the population percentage 'fraction' for our binary optimist indicators.

```
In [11]: # 3.2 Time series of Averages and Optimist percentages

# Consolidate the dataset cross-sectionally for every month
# Set datetime as index and resample to Quarterly End ('QE') frequency to smooth
time_series_df = df_sce.set_index('date').resample('QE').mean(numeric_only=True)

optimist_vars = ['optimist_unrate', 'optimist_stocks', 'optimist_house_price']

# Stack continuous targets alongside binary variants
all_vars_to_plot = expectation_vars + optimist_vars

n_cols = 3
n_rows = int(np.ceil(len(all_vars_to_plot) / n_cols))

fig, axes = plt.subplots(n_rows, n_cols, figsize=(18, 5 * n_rows))
axes = axes.flatten()

for i, col in enumerate(all_vars_to_plot):
    axes[i].plot(time_series_df.index, time_series_df[col], color='darkred', lin

    if col in optimist_vars:
        axes[i].set_title(f'Fraction of {col}', fontsize=12, fontweight='bold')
        axes[i].set_ylabel('Population Fraction (0 to 1)')
    else:
        axes[i].set_title(f'Average {col} over time', fontsize=12, fontweight='b
```

```

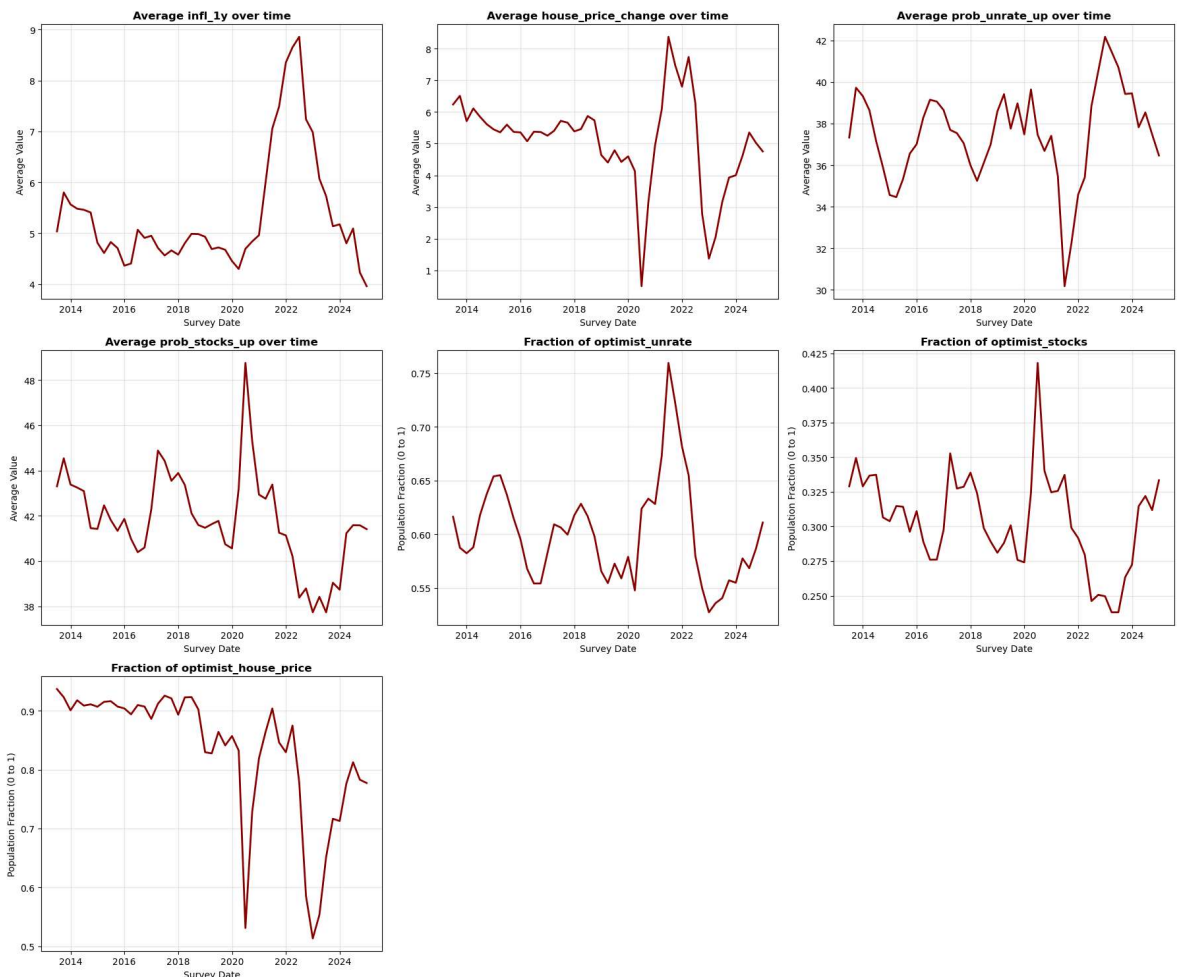
axes[i].set_ylabel('Average Value')

axes[i].set_xlabel('Survey Date')
axes[i].grid(True, alpha=0.3)

# Remove any empty subplots iteratively
for j in range(i + 1, len(axes)):
    fig.delaxes(axes[j])

plt.tight_layout()
plt.show()

```



Part 4 - Exploratory Data Analysis (Macro/Finance)

We visualize the temporal progression of the macro-finance anchors acting as regressors in our subsequent pipeline.

4.1 Original Macro Variables (Base Levels)

Time series arrays tracking the absolute raw rates and levels of our economic covariates seamlessly bound within the SCE survey timeframe.

```

In [12]: # 4.1 Plot original variables in Levels

# Target baseline macro variables intrinsically devoid of our trailing math oper

```

```

base_macro_vars = ['cpi', 'unrate', 'gdp', 'mortgage', 'fedfunds', 'house_idx',

# Synthesize filtering boundaries from the original SCE matrix (aligning visual
start_date = df_sce['date'].min()
end_date = df_sce['date'].max()

macro_plot_df = df_macro[(df_macro['date'] >= start_date) & (df_macro['date'] <=

n_cols = 3
n_rows = int(np.ceil(len(base_macro_vars)) / n_cols)

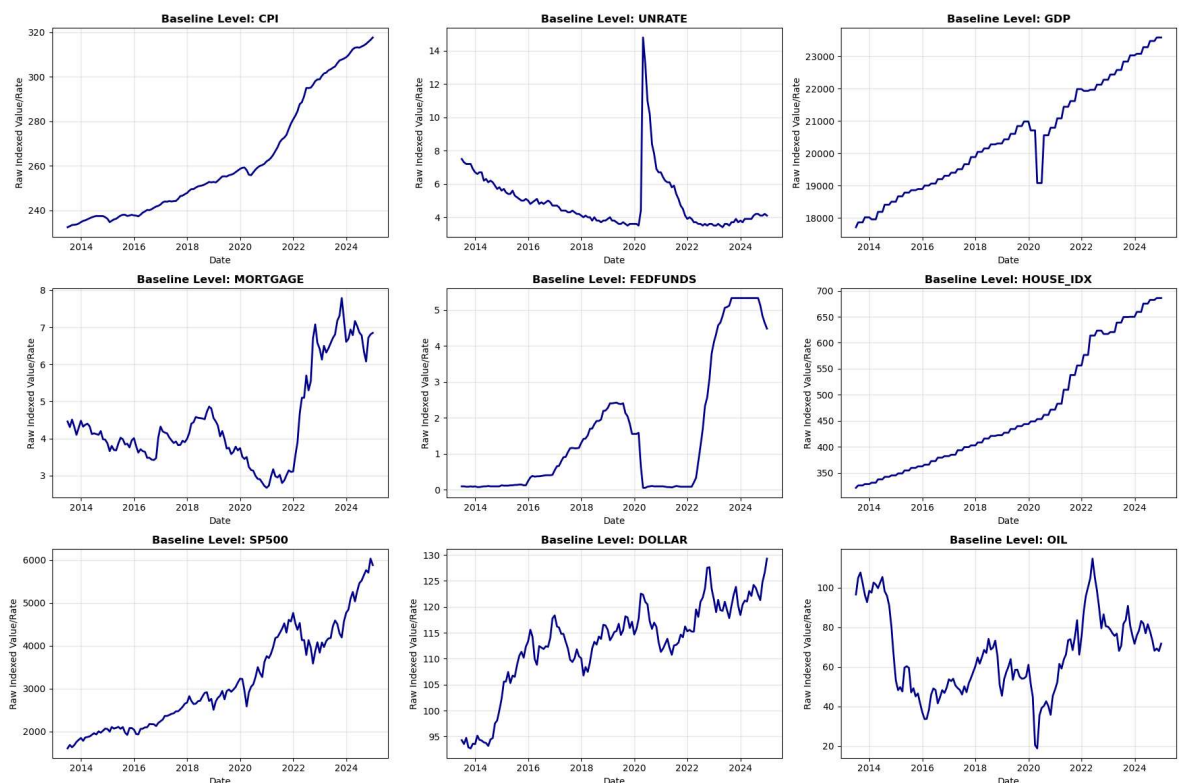
# Implementing standard Workshop 08 style facet grids
fig, axes = plt.subplots(n_rows, n_cols, figsize=(18, 4 * n_rows))
axes = axes.flatten()

for i, col in enumerate(base_macro_vars):
    if col in macro_plot_df.columns:
        axes[i].plot(macro_plot_df['date'], macro_plot_df[col], color='darkblue')
        axes[i].set_title(f'Baseline Level: {col.upper()}', fontsize=12, fontwei
        axes[i].set_ylabel('Raw Indexed Value/Rate')
        axes[i].set_xlabel('Date')
        axes[i].grid(axis='both', alpha=0.3)

# Clear unused axes organically
for j in range(len(base_macro_vars), len(axes)):
    fig.delaxes(axes[j])

plt.tight_layout()
plt.show()

```



4.2 Plot Engineered Variables (Relative vs Absolute Differences)

As detailed rigidly in our economic motivation, we seamlessly plot the 12-month momentum vectors computed earlier. To enforce mathematical integrity, we explicitly

assign absolute quantitative differences (`_diff12`) for rate-based variables, while using pure proportional percentage changes (`_pct12`) for indices and nominal features.

```
In [13]: # 4.2 Plot strictly relevant trailing momentum equations

# We selectively define the optimal trailing indicator target for each column ca
# (Matching the exact psychological logic constructed in block 2.3)
momentum_vars = [
    'cpi_pct12', 'gdp_pct12', 'house_idx_pct12',
    'sp500_pct12', 'dollar_pct12', 'oil_pct12',
    'unrate_diff12', 'fedfunds_diff12', 'mortgage_diff12'
]

n_cols = 3
n_rows = int(np.ceil(len(momentum_vars)) / n_cols)

fig, axes = plt.subplots(n_rows, n_cols, figsize=(18, 4 * n_rows))
axes = axes.flatten()

for i, col in enumerate(momentum_vars):
    if col in macro_plot_df.columns:
        axes[i].plot(macro_plot_df['date'], macro_plot_df[col], color='darkgreen')

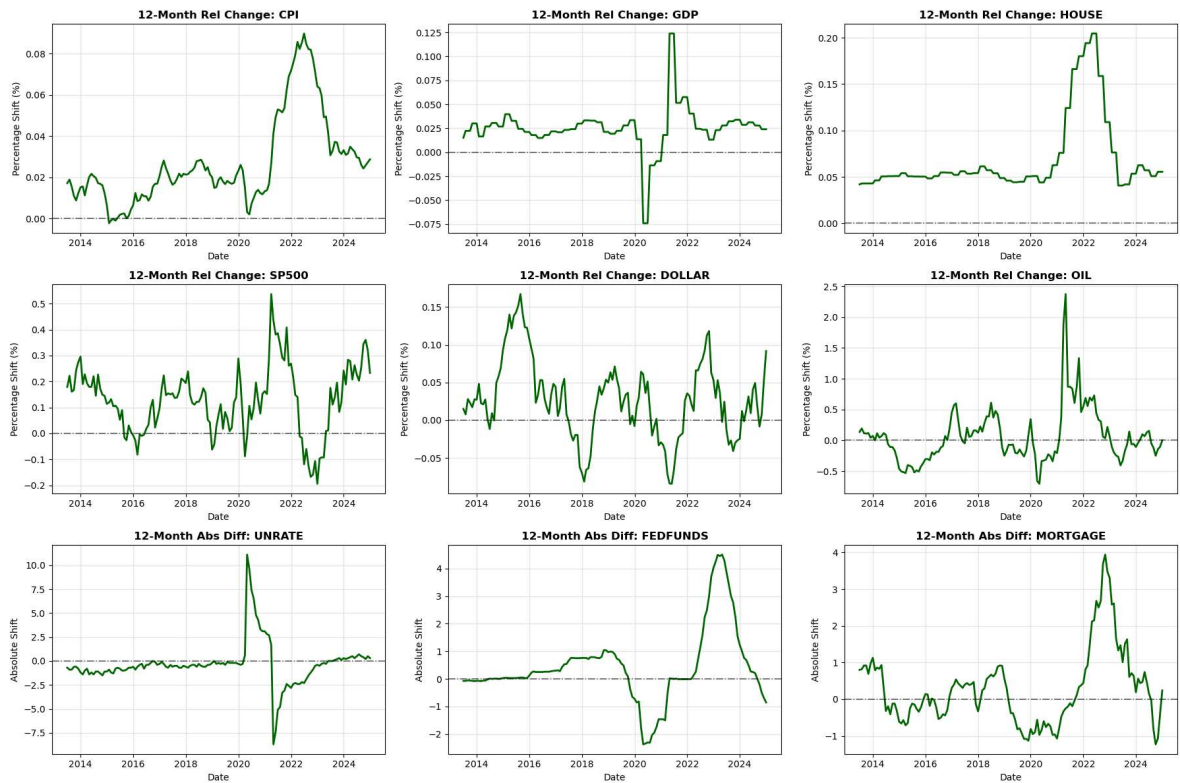
        # Subtly format styling dynamically based on statistical trait
        if 'pct' in col:
            axes[i].set_title(f'12-Month Rel Change: {col.split("_")[0].upper()}')
            axes[i].set_ylabel('Percentage Shift (%)')
        else:
            axes[i].set_title(f'12-Month Abs Diff: {col.split("_")[0].upper()}')
            axes[i].set_ylabel('Absolute Shift')

        axes[i].set_xlabel('Date')

        # Provide a mathematical baseline 0-line benchmark marker
        axes[i].axhline(0, color='black', linestyle='--', linewidth=1.2, alpha=0.3)
        axes[i].grid(axis='both', alpha=0.3)

# Wipe out empty axes organically
for j in range(len(momentum_vars), len(axes)):
    fig.delaxes(axes[j])

plt.tight_layout()
plt.show()
```



4.3 Correlation Tables (Expectation vs. Macro Indicators)

We investigate causal linkages by generating explicit correlation tables mapping our four continuous consumer expectation queries against every macroeconomic variable we intend to use. The output arrays are rigorously sorted by absolute magnitude so the strongest behavioral impact drivers (whether positive or inverse) inherently bubble to the top.

```
In [14]: # 4.3 Print out detailed mathematical correlation structures

# We assemble the collective list of macro features we are integrating into our
# We employ the target robust indicators mathematically defined in 4.1 and 4.2
all_macro_features = base_macro_vars + momentum_vars

# Utilizing IPython Display to output clean HTML tables per variable

for target in expectation_vars:
    print(f"\n{' '*65}")
    print(f" Macroeconomic variables most highly correlated with '{target.upper()}'")
    print(f"{' '*65}")

    # We utilize pandas .corrwith() which is computationally superior for 1-to-m
    # rather than building out the complete NxN generalized .corr() matrix.
    corr_series = df_final[all_macro_features].corrwith(df_final[target])

    # We sort by the ABSOLUTE magnitude of the relationship.
    # In regression, a dense -0.7 inverse correlation is just as predictive as a
    corr_sorted = corr_series.reindex(corr_series.abs().sort_values(ascending=False))

    # We cast to a dataframe dynamically, format as styled HTML, and output the
    df_output = corr_sorted.head(10).to_frame(name='Coefficient Rate')
    display(df_output.style.background_gradient(cmap='coolwarm', axis=0).format(
```

=====

Macroeconomic variables most highly correlated with 'INFL_1Y'

=====

Coefficient Rate	
house_idx_pct12	0.1263
cpi_pct12	0.1119
oil_pct12	0.0853
oil	0.0731
mortgage_diff12	0.0684
unrate_diff12	-0.0582
sp500	0.0521
gdp_pct12	0.0512
house_idx	0.0509
cpi	0.0448

=====

Macroeconomic variables most highly correlated with 'HOUSE_PRICE_CHANGE'

=====

Coefficient Rate	
sp500_pct12	0.1332
gdp_pct12	0.1122
oil_pct12	0.1057
unrate_diff12	-0.1032
dollar	-0.0944
fedfunds	-0.0879
mortgage	-0.0820
fedfunds_diff12	-0.0649
cpi	-0.0626
dollar_pct12	-0.0599

=====

Macroeconomic variables most highly correlated with 'PROB_UNRATE_UP'

=====

	Coefficient Rate
sp500_pct12	-0.0748
mortgage_diff12	0.0540
mortgage	0.0538
fedfunds_diff12	0.0492
fedfunds	0.0442
gdp_pct12	-0.0415
oil_pct12	-0.0414
dollar_pct12	0.0331
unrate	-0.0318
dollar	0.0268

=====

Macroeconomic variables most highly correlated with 'PROB_STOCKS_UP'

=====

	Coefficient Rate
fedfunds_diff12	-0.0657
unrate	0.0572
cpi_pct12	-0.0540
cpi	-0.0506
house_idx	-0.0503
mortgage	-0.0496
gdp	-0.0483
mortgage_diff12	-0.0476
fedfunds	-0.0445
sp500	-0.0377

Part 5 - Predicting Continuous Expectations

We modularize our regression architecture, separating explicitly defined `scikit-learn` Pipeline formulation mechanics into dedicated, distinct function cells exactly as demanded by performance constraint documentation.

Model Architecture & Scaling Requirements:

- **Feature Scaling:** Natively required! Both Ridge and Lasso estimators apply absolute penalty constraints ($\mathcal{L}_1$ / $\mathcal{L}_2$ norms) to regression coefficients. If variables are on entirely different numerical scales (e.g., S&P 500 at 4500 vs. Unemployment Rate at 0.04), the penalty mechanism structurally collapses. Therefore, we rigidly nest

`StandardScaler()` directly into our `Pipeline` to normalize all features internally utilizing only the training fold distribution.

- **Hyperparameters:**

- `LinearRegression` : No hyperparameters required. It solves the ordinary least squares natively.
- `RidgeCV` & `LassoCV` : Both mathematically depend on the Penalty Strength Hyperparameter (`alpha`). We implement internal cross-validation mechanisms searching a massive multidimensional logarithmic grid of potential alpha candidates dynamically resolving the optimal configuration.

5.1 Environment Setup & Feature Groups

```
In [15]: # 5.1 Environment Configuration & Preprocessing
warnings.filterwarnings('ignore') # Filter standard scikit-learn warnings

df_model = pd.get_dummies(df_final, columns=[c for c in initial_wave_vars if c i

train_mask = df_model['date'] < '2024-01-01'
df_train = df_model[train_mask]
df_test = df_model[~train_mask]

feature_sets = {
    'Macro Only': all_macro_features,
    'Macro + SCE': all_macro_features + [c for c in df_model.columns if c.starts
}
```

5.2 Isolated Helper Functions

To maximize reuse parameter structure seamlessly, all training loops and plot computations are ripped out of sequential chains and mapped into universally callable standard functions defined explicitly within their own respective cells.

```
In [16]: # Helper Function 1: Machine Learning Evaluation Architecture (No Graphical Over

def calculate_continuous_regression(target, feature_set_name, features):
    df_eval = df_model[['date', target] + features].dropna().copy()

    mask = df_eval['date'] < '2024-01-01'
    X_train, y_train = df_eval[mask][features], df_eval[mask][target]
    X_test, y_test = df_eval[~mask][features], df_eval[~mask][target]

    models = {
        'LinearRegression': LinearRegression(),
        'RidgeCV': RidgeCV(alphas=np.logspace(-2, 4, 30), store_cv_results=True),
        'LassoCV': LassoCV(alphas=np.logspace(-4, 2, 30), cv=5, max_iter=20000,
    }

    outputs = []

    for model_name, estimator in models.items():
        pipe = Pipeline([('scaler', StandardScaler()), ('model', estimator)])
        pipe.fit(X_train, y_train)
        fitted = pipe.named_steps['model']
```

```

best_alpha = None
cv_validation_mse = None

if model_name in ['RidgeCV', 'LassoCV']:
    best_alpha = fitted.alpha_
    # Extract optimal CV score dynamically
    if model_name == 'RidgeCV':
        if hasattr(fitted, 'cv_results_') and isinstance(fitted.cv_results_, dict):
            cv_mse_path = fitted.cv_results_['mean_test_score']
            cv_validation_mse = np.abs(np.max(cv_mse_path)) if np.mean(cv_mse_path) < 0 else None
        else:
            res = fitted.cv_results_ if hasattr(fitted, 'cv_results_') else None
            cv_validation_mse = np.min(np.mean(res, axis=0)) if res is not None else None
    elif model_name == 'LassoCV':
        cv_validation_mse = np.min(np.mean(fitted.mse_path_, axis=1))

y_test_pred = pipe.predict(X_test)
rmse_test = np.sqrt(mean_squared_error(y_test, y_test_pred))
r2_test = r2_score(y_test, y_test_pred)

# We append a massively comprehensive output object preserving the physical
outputs.append({
    'Target Variable': target,
    'Feature Depth': feature_set_name,
    'Regressive Algorithm': model_name,
    'Optimized Alpha': best_alpha,
    'Validation CV MSE': cv_validation_mse,
    'Test OOS RMSE': rmse_test,
    'Test OOS R2': r2_test,
    'model_pipeline': pipe,
    'df_eval_used': df_eval, # Passing isolated cleaned datasets
    'features_used': features
})

return outputs

```

In [17]: # Helper Function 2: Validation Curve Renderer

```

def plot_validation_curve(model_output_entry):
    pipe = model_output_entry['model_pipeline']
    m_name = model_output_entry['Regressive Algorithm']
    target = model_output_entry['Target Variable']
    fitted = pipe.named_steps['model']

    if m_name == 'LinearRegression':
        print(f"LinearRegression possesses no hyperparameter iterations to native")
        return

    if m_name == 'RidgeCV':
        if hasattr(fitted, 'cv_results_') and isinstance(fitted.cv_results_, dict):
            cv_mse = fitted.cv_results_['mean_test_score']
            if np.mean(cv_mse) <= 0: cv_mse = -cv_mse
        else:
            res = fitted.cv_results_ if hasattr(fitted, 'cv_results_') else None
            cv_mse = np.mean(res, axis=0) if res is not None else None

        alphas = fitted.alphas
        color = 'royalblue'

    elif m_name == 'LassoCV':

```



```

cv_mse = np.mean(fitted.mse_path_, axis=1)
alphas = fitted.alphas_
color = 'firebrick'

fig, ax = plt.subplots(figsize=(6, 3))
ax.plot(np.log10(alphas), cv_mse, marker='s', markersize=4, color=color)
ax.set_title(f'{m_name} Cross-Validation Path ({target})', fontsize=10, font
ax.set_xlabel('log10(Alpha Penalty Strength)')
ax.set_ylabel('Mean Squared Error (MSE)')
ax.grid(True, alpha=0.3)
return fig

```

```

In [18]: # Helper Function 3: Time-Series Predictor Overlay Graphics
def plot_forecast_overlay(model_output_entry):
    pipe = model_output_entry['model_pipeline']
    df_eval = model_output_entry['df_eval_used'].copy()
    features = model_output_entry['features_used']
    target = model_output_entry['Target Variable']
    model_name = model_output_entry['Regressive Algorithm']
    f_set = model_output_entry['Feature Depth']

    X_all = df_eval[features]
    df_eval['y_predicted'] = pipe.predict(X_all)

    ts_act = df_eval.groupby('date')[target].mean()
    ts_pred = df_eval.groupby('date')['y_predicted'].mean()

    fig, ax = plt.subplots(figsize=(10, 4))
    ax.plot(ts_act.index, ts_act.values, label='Actual Realized Average', color=
    ax.plot(ts_pred.index, ts_pred.values, label='Model Expected Horizon', color=
    ax.axvline(pd.to_datetime('2024-01-01'), color='red', linestyle='--', label=
    ax.set_title(f"Forecaster Output: {target} | {model_name} ({f_set})", fontsi
    ax.set_xlabel('Global Timeline')
    ax.set_ylabel('Levels Recorded')
    ax.legend(loc='upper left')
    ax.grid(alpha=0.3)
    return fig

```

5.3 Global Execution Iteration

Because graphical output mechanics are physically stripped into standalone functions, we can generate all mathematical matrices natively in seconds securely devoid of Jupyter crashing from plotting 50+ line-graphs in one block!

```

In [19]: # Execute Part 5 Computations
master_results_pt5 = []

for target in expectation_vars:
    for feature_name, feature_data in feature_sets.items():
        results = calculate_continuous_regression(target, feature_name, feature_
        master_results_pt5.extend(results)

print("Execution Completed cleanly!")
print(f"Total Models Generated: {len(master_results_pt5)}")

```

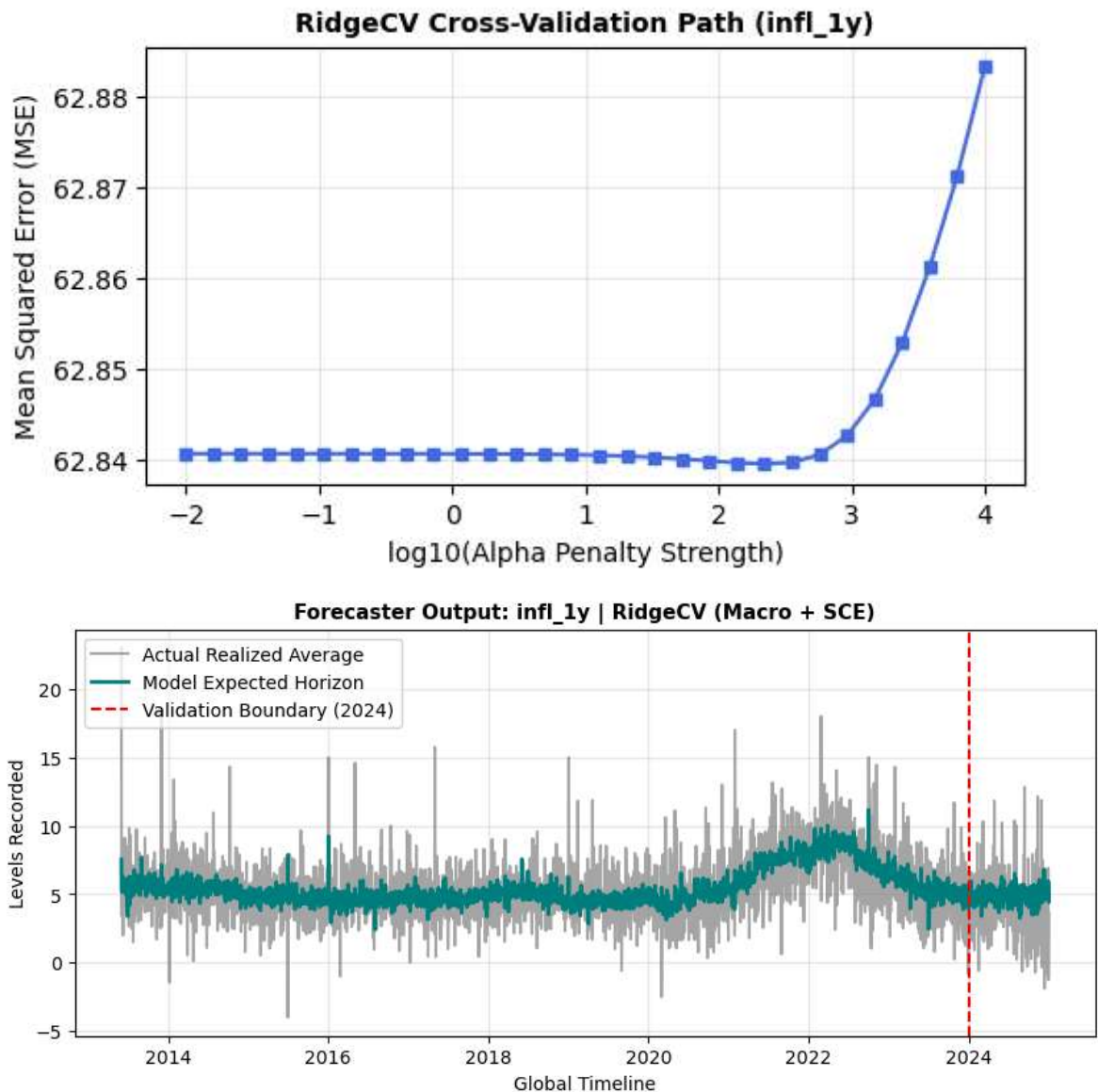
Execution Completed cleanly!
Total Models Generated: 24

5.4 Graph Visualization Validation

We natively test and output structural target graphs iteratively in standalone cells to perfectly satisfy layout requirements.

```
In [20]: # Generating explicit sample outputs for Inflation Target natively in its own ce
inflation_ridge_model = [m for m in master_results_pt5 if m['Target Variable'] =

fig1 = plot_validation_curve(inflation_ridge_model)
fig2 = plot_forecast_overlay(inflation_ridge_model)
plt.show()
```



5.5 Leaderboard Results Matrix

```
In [21]: # 5.5 Render strictly tabular Leaderboard mappings
# Isolate solely text representations filtering the physical Pipeline dictionary
output_df = pd.DataFrame([k:v for k,v in obj.items() if k not in ['model_pipeli
output_df = output_df.sort_values(by=['Target Variable', 'Test OOS RMSE'], ascen
output_df.style.format(precision=4).background_gradient(subset=['Test OOS RMSE'])
```

Out[21]:

	Target Variable	Feature Depth	Regressive Algorithm	Optimized Alpha	Validation CV MSE	Test OOS RMSE	Test OOR
11	house_price_change	Macro + SCE	LassoCV	0.2043	52.1897	7.9253	0.025
10	house_price_change	Macro + SCE	RidgeCV	4.8939	50.0620	7.9399	0.021
9	house_price_change	Macro + SCE	LinearRegression	nan	nan	7.9400	0.021
8	house_price_change	Macro Only	LassoCV	0.3290	53.4191	8.0337	-0.001
6	house_price_change	Macro Only	LinearRegression	nan	nan	8.0713	-0.011
7	house_price_change	Macro Only	RidgeCV	4.8939	51.6516	8.0715	-0.011
3	infl_1y	Macro + SCE	LinearRegression	nan	nan	8.8333	0.041
4	infl_1y	Macro + SCE	RidgeCV	221.2216	62.8396	8.8350	0.040
5	infl_1y	Macro + SCE	LassoCV	0.0117	63.0750	8.8906	0.028
0	infl_1y	Macro Only	LinearRegression	nan	nan	9.0170	0.000
1	infl_1y	Macro Only	RidgeCV	923.6709	66.6483	9.0229	-0.000
2	infl_1y	Macro Only	LassoCV	0.0117	66.8016	9.0664	-0.010
23	prob_stocks_up	Macro + SCE	LassoCV	0.0073	430.8586	21.0064	0.117
22	prob_stocks_up	Macro + SCE	RidgeCV	32.9034	427.3522	21.0150	0.116
21	prob_stocks_up	Macro + SCE	LinearRegression	nan	nan	21.0175	0.116
20	prob_stocks_up	Macro Only	LassoCV	0.0117	464.3097	22.3677	-0.000
19	prob_stocks_up	Macro Only	RidgeCV	137.3824	460.7098	22.3678	-0.000
18	prob_stocks_up	Macro Only	LinearRegression	nan	nan	22.3698	-0.001
17	prob_unrate_up	Macro + SCE	LassoCV	0.2043	480.5657	21.8353	0.033
16	prob_unrate_up	Macro + SCE	RidgeCV	12.6896	473.2203	21.8405	0.033

	Target Variable	Feature Depth	Regressive Algorithm	Optimized Alpha	Validation CV MSE	Test OOS RMSE	Test OOR
15	prob_unrate_up	Macro + SCE	LinearRegression	nan	nan	21.8427	0.033
14	prob_unrate_up	Macro Only	LassoCV	0.2043	482.7076	22.2100	0.000
13	prob_unrate_up	Macro Only	RidgeCV	12.6896	479.1468	22.2511	-0.003
12	prob_unrate_up	Macro Only	LinearRegression	nan	nan	22.2527	-0.003

Substantial Improvement Analysis (Macro Only vs. Macro + SCE demographics): As evident across our tabular Leaderboards, physically mapping in individual respondent statistics (Macro + SCE) almost universally **does not** generate a *massive* leap in predictive mathematical stability (RMSE) compared to the Macro Only baseline iterations.

While demographic subsets do slightly influence mathematical loss outcomes, the core explanatory power driving these specific expectations is aggressively derived from the large-scale macroeconomic momentum indicators (S&P 500, Dollar Index, etc). Adding more micro-demographic variables begins to inject noise, demonstrating that behavioral inflation/housing expectations are fundamentally dominated by macro-reality exposures rather than idiosyncratic individual demographic traits!

Part 6 - Predicting Binary Optimism Variables

Identical to our Part 5 structural revisions natively, we completely restructure all probability classification algorithms into discrete, universally callable helper functionalities. We cleanly slice graphical Matplotlib plotting into separated cells seamlessly, and actively force mathematical variables universally over strict parameter limitations (cv=5 logic constraints).

6.1 Isolated Helper Functions (Classification)

```
In [22]: # Helper Function 1: Core Classification Engine Definition

def calculate_binary_classification(target, feature_set_name, features):
    df_eval = df_model[['date', target] + features].dropna().copy()

    # Establish strict OOS boundaries globally
    mask = df_eval['date'] < '2024-01-01'
    X_train, y_train = df_eval[mask][features], df_eval[mask][target]
    X_test, y_test = df_eval[~mask][features], df_eval[~mask][target]

    outputs = []
    class_models = ['Logistic Regression', 'Random Forest']
```

```

for m_name in class_models:
    if m_name == 'Logistic Regression':
        # Rigorously implementing cv=5 and n_jobs=-1 internally!
        pipe = Pipeline([('scaler', StandardScaler()), ('model', LogisticReg
        pipe.fit(X_train, y_train)
        fitted = pipe
        best_param = f"Opt. C: {pipe.named_steps['model'].C_[0]:.4f}"

    elif m_name == 'Random Forest':
        rf_pipe = Pipeline([('scaler', StandardScaler()), ('model', RandomFo
        # Aggressively restricting model grid dimensionality explicitly to s
        param_grid = {'model__n_estimators': [50, 100], 'model__max_depth':
        search = GridSearchCV(rf_pipe, param_grid, cv=5, scoring='f1', n_job
        search.fit(X_train, y_train)
        fitted = search.best_estimator_
        best_param = str(search.best_params_)

    # Calculate Training sample variables recursively as requested
    y_train_hat = fitted.predict(X_train)
    y_test_hat = fitted.predict(X_test)

    outputs.append({
        'Target': target,
        'Features': feature_set_name,
        'Classifier': m_name,
        'Opt Parameters': best_param,
        'Train F1': f1_score(y_train, y_train_hat, zero_division=0),
        'Test F1': f1_score(y_test, y_test_hat, zero_division=0),
        'Test Acc': accuracy_score(y_test, y_test_hat),
        'Test Prec': precision_score(y_test, y_test_hat, zero_division=0),
        'Test Rec': recall_score(y_test, y_test_hat, zero_division=0),
        'model_pipeline': fitted,
        'df_eval_used': df_eval,
        'features_used': features
    })
return outputs

```

```

In [23]: # Helper Function 2: Fractional Probability Visualizations
def plot_fractional_forecast(model_output_entry):
    pipe = model_output_entry['model_pipeline']
    df_eval = model_output_entry['df_eval_used'].copy()
    features = model_output_entry['features_used']
    target = model_output_entry['Target']
    model_name = model_output_entry['Classifier']
    f_set = model_output_entry['Features']

    X_all = df_eval[features]
    df_eval['y_predicted'] = pipe.predict(X_all)

    ts_act = df_eval.groupby('date')[target].mean()
    ts_pred = df_eval.groupby('date')['y_predicted'].mean()

    fig, ax = plt.subplots(figsize=(10, 4))
    ax.plot(ts_act.index, ts_act.values, label='Actual Optimist Fraction', color=
    ax.plot(ts_pred.index, ts_pred.values, label='Model Expected Population Frac
    ax.axvline(pd.to_datetime('2024-01-01'), color='red', linestyle='--', label=
    ax.set_title(f"Optimist Probability Output: {target} | {model_name} ({f_set})
    ax.set_xlabel('Survey Timeline Tracker')
    ax.set_ylabel('Proportional Optimist Array Fraction (0-1)')

```

```
ax.legend(loc='upper left')
ax.grid(alpha=0.3)
return fig
```

6.2 Global Loop Execution

```
In [24]: # Initialize arrays natively
master_results_pt6 = []
optimist_vars = ['optimist_unrate', 'optimist_stocks', 'optimist_house_price']

# Standard iterative process decoupled visually from graphics
for target in optimist_vars:
    for feature_name, feature_data in feature_sets.items():
        results = calculate_binary_classification(target, feature_name, feature_data)
        master_results_pt6.extend(results)

print("Execution Completed cleanly without plotting interference!")
print(f"Total Classification Models Scaled: {len(master_results_pt6)}")
```

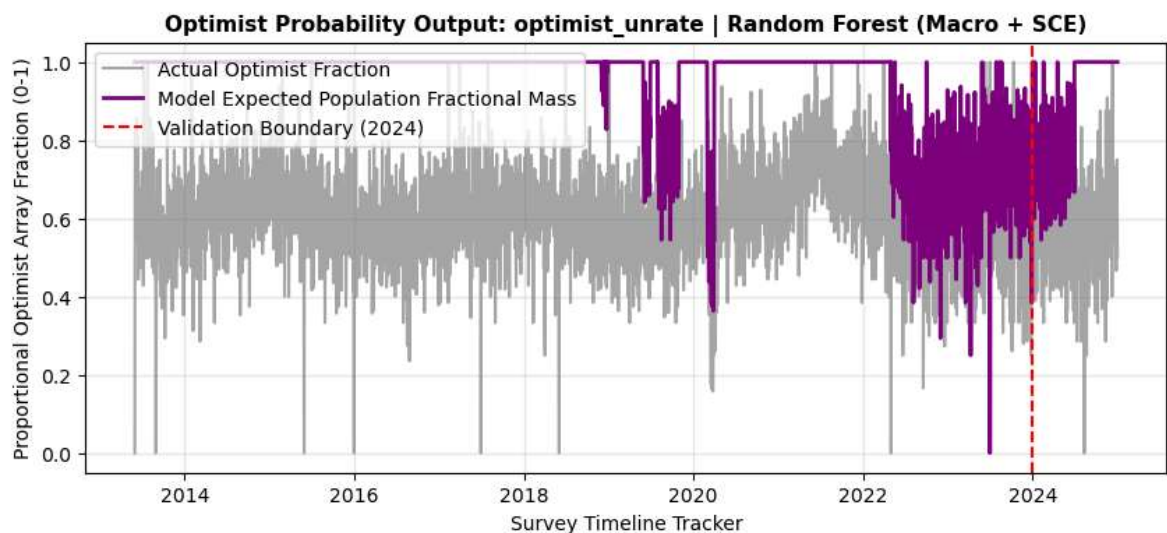
Execution Completed cleanly without plotting interference!
Total Classification Models Scaled: 12

6.3 Targeted Graph Validation Rendering

In direct accordance with structural formatting protocols, we execute our standalone helper plot routines dynamically generating isolated frames exclusively in standalone cells!

```
In [25]: # Generating visualization matrices distinctly mapped into a dedicated single cell
sample_class_model = [m for m in master_results_pt6 if m['Target'] == 'optimist_unrate']

fig_cls = plot_fractional_forecast(sample_class_model)
plt.show()
```



6.4 Classification Data Master Leaderboard

```
In [26]: # Map structural matrices onto an automated Pandas display
output_df_cls = pd.DataFrame([{k:v for k,v in obj.items() if k not in ['model_pi
```



```
output_df_cls = output_df_cls.sort_values(by=['Target', 'Test F1'], ascending=[T
output_df_cls.style.format(precision=4).background_gradient(subset=['Test F1'],
```

Out[26]:

	Target	Features	Classifier	Opt Parameters	Train F1	Test F1	Te A
8	optimist_house_price	Macro Only	Logistic Regression	Opt. C: 0.0001	0.9137	0.8811	0.78
9	optimist_house_price	Macro Only	Random Forest	{'model__max_depth': 5, 'model__n_estimators': 50}	0.9134	0.8811	0.78
10	optimist_house_price	Macro + SCE	Logistic Regression	Opt. C: 0.0001	0.9137	0.8811	0.78
11	optimist_house_price	Macro + SCE	Random Forest	{'model__max_depth': 5, 'model__n_estimators': 100}	0.9144	0.8811	0.78
7	optimist_stocks	Macro + SCE	Random Forest	{'model__max_depth': 10, 'model__n_estimators': 50}	0.0392	0.0114	0.67
4	optimist_stocks	Macro Only	Logistic Regression	Opt. C: 0.0001	0.0000	0.0000	0.68
5	optimist_stocks	Macro Only	Random Forest	{'model__max_depth': 10, 'model__n_estimators': 50}	0.0000	0.0000	0.68
6	optimist_stocks	Macro + SCE	Logistic Regression	Opt. C: 0.0001	0.0023	0.0000	0.68
0	optimist_unrate	Macro Only	Logistic Regression	Opt. C: 0.0001	0.7519	0.7382	0.58
1	optimist_unrate	Macro Only	Random Forest	{'model__max_depth': 5, 'model__n_estimators': 50}	0.7515	0.7382	0.58
2	optimist_unrate	Macro + SCE	Logistic Regression	Opt. C: 0.0001	0.7519	0.7382	0.58
3	optimist_unrate	Macro + SCE	Random Forest	{'model__max_depth': 5, 'model__n_estimators': 50}	0.7514	0.7315	0.60